

Two books, one exchange

Axe is connected and measured. Their live book is **fourteen thousand open loans against Micromart Fintech's seventy-nine** — which makes Axe the anchor contributor, not the guest. What remains before a query between them is real is a short list of changes on your servers, and it is shorter than it looks: the Interchange needs **no inbound port on either lender's machine**.

What is true this morning

Both ServiceSuite servers were read over Tailscale while writing this. Figures are rounded; the shape is what matters.

ENTITY	SERVER	BORROWERS	OPEN LOANS	OUTSTANDING	LAST DISBURSEMENT
3003 · Axe — Boresha	axemicro	43,000	14,000	KES 122.8M	25 Aug 2026
3004 · Axe — Stawi	axemicro	417	115	KES 8.5M	24 Jul 2026
3002 · Micromart Africa	services	142,000	59,000	KES 347.5M	25 Aug 2026
3003 · Micromart Check off	services	2,000	4,000	KES 499.4M	25 Aug 2026
3005 · Micromart Fintech	services	17,000	79	KES 818k	25 Aug 2026

READ THIS ROW TWICE

Entity 3003 is “Micromart Check off” on one server and “Axe — Boresha” on the other. EntityIds are unique within a ServiceSuite instance and nowhere else, and the Registry’s existing member codes are built from them — KE/LENDER/3002 , KE/LENDER/3005 . A bare KE/LENDER/3003 would therefore name two different lenders, and whichever registered first would silently own the other’s exposure.

Axe is registered today as KE/LENDER/AXE-3003 and KE/LENDER/AXE-3004 to step around it. That is a patch, not a fix — see the decisions at the end.

Where Sprint 3 actually stands

Two sprint ladders have been running under the same number, which is worth separating before anything is called done. Blueprint v2 Table 7 numbers the *Interchange* sprints; the customer-app build plan numbers the *PWA* sprints. They are not the same list, and only one of them has a measurable acceptance test.

BLUEPRINT V2 · TABLE 7	DONE WHEN	STATE
2 · Registry, OPRF, policy engine, metering, hash-chained log	Two members exchange a signed, consented, logged call	MET
3 · Exposure engine — fan-out, Bloom filters, aggregation. CRB 2.0 types 1, 2, 20	p95 under 400 ms on the real Micromart book	ENGINE YES, BOOK NO

Every component of Sprint 3 exists and is tested: the broker fans out in parallel with per-member timeouts, the Bloom screening has a no-false-negative check, the CRB 2.0 envelopes are built, and `verify:exposure` measures p95 over thirty samples. What none of it had was a book. The Registry holds fifteen members, all with keys — and **zero holdings and zero published filters**. The 400 ms was measured against four hundred invented borrowers.

THE HONEST READING

Sprint 3’s done-when says “*on the real Micromart book*”, and that clause is the whole sprint. Everything else was finished weeks ago. The gap was never the algorithm — it was that nothing had ever pointed the engine at Serviceconnect.

The customer app’s own Sprint 3.0 (“Connect the Interchange”) is downstream of exactly the same gap, which is why the credit-file screen still says *not switched on*. One piece of work closes both.

What was built today to close it

- **A node ingest** that reads one entity’s live book, derives a `subject_token` per borrower through the blinded OPRF exchange, drops the identifiers, and publishes tokens plus four aggregates. It runs in the suite deployment — which already holds the credentials and the tailnet reach — so no lender’s SQL password is copied into the Registry repo.
- **Generational publication.** A fourteen-thousand-row book takes several requests. The obvious implementation, delete-then-insert, opens a window in which the member truthfully answers *no exposure* for every borrower they hold. Chunks are written under a new generation and become visible in one statement at the end.
- **Batch tokenisation**, metered per element rather than per request, with a separate `INGEST` allowance from the `SERVING` one — a member walking their own book and a member walking the national ID space look identical to the Registry, so the two budgets are granted, not inferred.
- **“Not published” is no longer “owes nothing.”** A member who has never run an ingest now answers `BOOK_NOT_PUBLISHED` , and the broker counts that as a non-response that marks the result `partial` . Silently clearing a borrower because a node was never ingested is the most dangerous answer this service can give.

What has to change on your servers

Less than you are expecting. Take the network question first, because it is the one you asked and the answer removes most of the work.

No, do not open a port or build a security group for the Interchange

RDP on 3389 needs a firewall rule and a security group because somebody outside is dialling in. The Interchange is the opposite shape: **nothing ever dials in to Axe's or Micromart's server**. The node runs on your hardware — blueprint v2 §1, “*members get an endpoint, not a server*” — and it dials *out* to SQL over Tailscale, then pushes to the Registry over HTTPS. Both connections are outbound from machines you already control.

Tailscale is not a hole in the firewall. It is an authenticated overlay: the listener is bound to the tailnet interface, every peer is a device you admitted, and the SQL port stays closed to the internet. That is already how Micromart works, and it is how Axe now works.

MEASURED WHILE WRITING THIS

100.103.154.73:4420 — Axe, over the tailnet — **open, 870 ms, answers a query.**

213.148.17.54:4420 — Axe, the public address in the portal config — **no answer.**

That is the correct posture and it should stay that way. The one thing worth confirming with whoever runs that box: if the public 4420 is open to any source at all, close it. A SQL Server exposed to the internet with a shared application login is a larger risk than anything on this page.

What does need doing: a login that can only read what the node reads

Today the suite connects as `SystemUser1` on Axe and `service_suite_user` on Micromart. Those are the application's own logins — they can write, and they can read every entity on the server. The node needs neither. Ten unrelated companies share the `.198` book separated by nothing but `EntityId`, so scope is a security boundary, not a filter.

1

Create one read-only login per server

Separate from the application login, so revoking Interchange access never takes the LMS down with it, and so the audit can tell the two apart.

```
-- On each ServiceSuite server, once.
CREATE LOGIN interchange_node
  WITH PASSWORD = '<generated, 32+ chars>', CHECK_POLICY = ON;

USE Serviceconnect;
CREATE USER interchange_node FOR LOGIN interchange_node;
```

2

Give it a view per entity, with the scope baked in

The `EntityId` lives inside the view, not in a parameter the caller supplies. A node key that leaked could then still only ever see the one book it was issued for — the scope is not something a request can widen.

```
CREATE SCHEMA interchange AUTHORIZATION dbo;
GO

-- One per member entity: 3003 and 3004 on Axe; 3002 and 3005 on Micromart.
CREATE VIEW interchange.BookExposure_3003 AS
SELECT b.NationalID                                AS NationalID,
       COUNT(1.ID)                                AS ActiveLoans,
       SUM(CAST(1.LoanBalance AS BIGINT))           AS OutstandingKes,
       MAX(1.BorrowDate)                           AS NewestDisbursedAt,
       MAX(DATEDIFF(day, 1.ExpectedClearDate, GETDATE())) AS WorstDpd
FROM   Loans l
JOIN   Borrowers b ON b.ID = l.BorrowerId AND b.EntityId = l.EntityId
WHERE  1.EntityId = 3003                          -- the scope, fixed here
       AND 1.isApproved = 1 AND 1.LoanCleared = 0 AND 1.LoanBalance > 0
GROUP BY b.NationalID;
```

3

Grant the schema, deny everything else

`DENY` beats a forgotten `GRANT` later — it is the rule that survives someone adding this user to a role in six months.

```
GRANT SELECT ON SCHEMA::interchange TO interchange_node;
DENY  SELECT, INSERT, UPDATE, DELETE ON SCHEMA::dbo TO interchange_node;
```

4

Hand over the connection string, not the password

It goes into the suite deployment’s secret store as `SERVICESUITE_CONN_AXE`, pointed at the *tailnet* address. The current entry uses the application login and should be swapped for `interchange_node` once the view exists; the ingest reads the base tables today and moves to the view in the same change.

ONE THING THE NODE GENUINELY NEEDS, AND WHY

The view returns `NationalID`. It has to: the OPRF has to blind a real identifier, and that can only happen inside the perimeter. It exists in memory for the length of one ingest and is never written, never logged and never sent — what leaves is a 128-character token that cannot be reversed, even by us, without the ecosystem key.

That is the sentence to say to a member, and it is the one the blueprint warns you to say precisely: not “your data never leaves you”, which stopped being true when you took on hosting the nodes, but “identity is destroyed before it reaches anything we store.”

The six systems, and the message you want gone

“Micromart’s server is not reachable right now.” It is not an outage and never was — it is topology, and it has one cause.

The launcher shows that banner when `isOrgConfigured()` is false, which is true when a deployment has **neither** a connection string it can route with **nor** a relay. Micromart's SQL Server lives on a Tailscale CGNAT address with no internet route at all, so on Vercel the connection string can be set perfectly and still connect to nothing. The relay exists precisely for this, and `npm run test:relay` reports `SERVICESUITE_RELAY_URL` and `SERVICESUITE_RELAY_SECRET` unset — so the relay path is not merely down, it has never been switched on.

- **Install Node on a tailnet host and run the relay there.** `servicesuite-in` (100.115.241.36) is online and is the intended home; this workstation, `arch-bishop`, already has Funnel enabled and can prove the path in ten minutes if you want it working before the install.
- **Expose it with Funnel, not a firewall rule.** `npm run relay`, then `tailscale funnel 8787`. That publishes one HTTPS endpoint that only answers HMAC-signed queries inside a two-minute window and refuses writes unless separately armed.
- **Set the two variables on Vercel and nowhere else.** With a relay configured, `SERVICESUITE_CONN_MICROMART` is not needed in the cloud at all — Micromart's database password never leaves the tailnet, and a compromise of the hosting account yields a secret that can ask a read-only relay for numbers already on a screen.
- **Then run `npm run test:relay` and `npm run test:live`** — the second is an outside-in check that separates *live* from *degraded* from *broken* on every subdomain, which is the actual round trip of the six systems.

Decisions that are yours

Four, and the first one gets harder to change every week it waits.

1 · What a member code is made of

Entity IDs are not globally unique and two of them already collide across your two servers. Codes built from them will keep colliding as the cohort grows, and a code is the thing consent, audit and the message log are all keyed to — it is the hardest field in the system to change later.

Recommendation: key member codes on the KRA PIN already sitting in `BsEntity.EntityTaxRef` — `KE/LENDER/A0KA070K0` for Axe Boresha. It is globally unique, it is what an X-Road member identifier is meant to be, and it is legible to a regulator. Migrate the fifteen existing members in one pass now, while the message log is short.

2 · How big an ingest allowance each member gets

The Registry cannot tell a member tokenising their own book from a member enumerating the national ID space, because by construction it never sees the identifiers. So the allowance is granted rather than inferred. It currently defaults to 250,000 elements a day per member, spent visibly under `kind=INGEST`.

Recommendation: set it per member at roughly twice their declared open book — Axe Boresha at 30,000, Micromart Africa at 120,000 — so a run that suddenly wants five times the book shows up in the audit as exactly that.

3 · How often a book is republished

Exposure is a real-time product built on a periodic ingest, so staleness is a property the answer carries: every node response now returns `as_of`. The blueprint’s filter cadence is fifteen minutes.

Recommendation: fifteen minutes for Axe Boresha and Micromart Africa, which disburse daily; hourly for Stawi and Fintech. Whatever you pick, put `as_of` on the customer-facing screen — a lender making a credit decision is entitled to know how old the answer is.

4 · Whether Axe queries on day one

A new member registers as `SHADOW`: contributing, not yet querying. That is the reciprocity rule enforced in the policy engine rather than in a contract, and it exists so nobody reads the ecosystem having published nothing.

Recommendation: leave both Axe entities in `SHADOW` until their first successful ingest commits, then promote. It costs a day and it makes the rule real rather than decorative — which matters more with the largest contributor than with the smallest.

Still open

ITEM	STATE	NEEDS
Registry schema migration — generational holdings, batch issuance	BLOCKED	Approval to run <code>prisma db push</code> against the hosted Postgres. The affected table is empty; the warning is about a replaced unique constraint, not data.

ITEM	STATE	NEEDS
Node on <code>servicesuite-in</code>	CARRIED	Install, then the relay steps above. This is what removes the launcher banner.
<code>PAYMENTS_ENTITY_ID = 7</code>	CARRIED	Confirmation from Micromart. Entity 7 404s on their config endpoint, so it is a different id space or it is wrong — and wrong means no STK prompt reaches the handset.
Black-on-black branding	CARRIED	Two rows now, not one: entity 3005 on Micromart and entity 3004 on Axe both carry <code>#000000</code> for primary and secondary.
<code>npm run demo:room</code> against the live org	NEXT	A run, once the relay is up.
Directory services still marked specified	NEXT	All nine, including <code>exposure-v1</code> , are flagged not-live in the Registry. Flip <code>exposure-v1</code> when the first two books have committed.